

UNIT - IV Feature Engineering Lecture 9 Hrs

Introduction, creating a feature set, Engineering features for ML applications, using rescaling techniques to improve the learnability of features, creating effective derived variables, reinterpreting non-numeric features, using feature selection techniques, performing feature selection, Feature engineering in practice, acquiring data via RESTful APIs, Testing the performance of our model, Twitter, Deriving and selecting variables using feature engineering techniques

Introduction

- Feature engineering plays a crucial role in machine learning by transforming raw data into meaningful features that enhance model performance. The phrase *"garbage in, garbage out"* is highly relevant here, emphasizing the importance of high-quality features.
- In this chapter focuses on a more general type of data—**quantitative and categorical data**—which is widely used in real-world applications.
- These types of data can be collected from diverse sources, such as:
 - Telemetry sensors in forests
 - Game consoles
 - Financial transactions
 - Geological surveys
 - Bioassay experiments
- Despite variations in data sources, the fundamental principles of feature engineering remain the same. This chapter aims to teach:
 1. **Feature set creation** and the limitations of feature engineering.
 2. **Techniques to improve datasets** through transformation and enhancement.
 3. **The role of domain knowledge** in feature engineering.
 4. **Evaluation of features** to retain only the most valuable ones.
- By the end of this chapter, you will gain hands-on experience in performing multiple iterations and specialized tests to assess the effectiveness of engineered features.

Creating a Feature Set

- The **quality of input data** is the most critical factor in successful machine learning. Even a well-optimized model cannot perform well if fed with misleading, poorly normalized, or uninformative data.
- In some cases, you may have control over the **data collection process** or access to large, diverse datasets. By leveraging the right knowledge and techniques, you can extract meaningful features to improve model performance.

Feature Set Design Process

- A structured approach to designing feature sets ensures that no valuable information is overlooked. In this chapter, we introduce:
 - **Techniques to extend or reinterpret existing features** to generate a broad set of useful parameters.
 - **Methods to assess feature effectiveness** while being cautious of potential risks in engineered datasets.

Engineering Features for ML Applications

- Once you have cleaned and validated the dataset, additional work is required before it can be used effectively in a machine learning model. This section explores methods for refining and enhancing data.

Rescaling Techniques for Better Learnability

- Many machine learning models, particularly those based on **gradient descent**, are sensitive to differences in feature scales. When datasets contain features with vastly different ranges, models may struggle to converge, leading to poor performance.

Gradient Descent and Scaling

- Gradient descent algorithms optimize models by iteratively adjusting weights to minimize an objective function or loss. However, if features are on different scales, the optimization process may be distorted. For example:

- **Unscaled data** results in highly irregular parameter spaces, making training inefficient.
- **Scaled data** ensures that all features contribute equally to the learning process.

Linear Rescaling (Normalization)

- One common rescaling technique is **linear rescaling**, which transforms feature values into a range between 0 and 1. The formula is:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- For instance, given a feature vector [0,10,25,20,18], its normalized form would be:
[0,0.4,1,0.8,0.72]
- This ensures that all features have comparable scales, improving model stability.

Nonlinear Scaling: Log Transformation

- In some cases, **log-scaling** is more suitable, especially for features exhibiting exponential growth patterns (e.g., financial data, population growth).
- Instead of maintaining linear relationships, log transformation adjusts relative spacing between values, making it particularly useful for handling:
 - **Highly skewed data** with extreme outliers.
 - **Power-law distributions**, where a small number of observations dominate the dataset.
- For example, consider a dataset containing individual net wealth statistics:

Statistic	Wealth (Before Log Scaling)
Min	1
1st Quartile	42.5
Mean	3,205,433.343
Median	600
3rd Quartile	1,358
Max	10,000,000,000

- Prior to rescaling, this population is hugely skewed toward that single individual with absurd net worth. The distribution of cases per decile is as follows:

Range	Count of Cases
0 > 0.1	3060
0.1 > 0.2	0
0.2 > 0.3	0
0.3 > 0.4	0
0.4 > 0.5	0
0.5 > 0.6	0
0.6 > 0.7	0
0.7 > 0.8	0
0.8 > 0.9	0
0.9 > 1	1

- Without rescaling, the model would be dominated by extreme values. However, applying a **log-10 transformation** reduces the impact of outliers, ensuring a more balanced distribution.

Range (Log Scaled)	Count of Cases
0 > 0.1	740
0.1 > 0.2	1633
0.2 > 0.3	544
0.6 > 0.7	0
0.3 > 0.4	141
0.7 > 0.8	1
0.4 > 0.5	0
0.8 > 0.9	0

Range (Log Scaled)	Count of Cases
0.5 > 0.6	1
0.9 > 1	1

- While log-scaling reduces the effect of extreme values, it can also introduce artificial separations among smaller values. Therefore, nonlinear transformations should only be applied when there is a valid reason, such as dealing with **exponential distributions**.

Creating Effective Derived Variables

- Apart from rescaling, another powerful feature engineering strategy involves **creating derived variables** that combine existing features to generate more meaningful information.

Common Derived Features

1. **Combinations of Two Variables**

- Multiplication, division, or normalization of one variable relative to another.
- Example: Converting speed and time into **distance** using $\text{Distance} = \text{Speed} \times \text{Time}$.

2. **Rate of Change Over Time**

- Useful for time-series data, where differences between time points reveal trends.
- Example: **Acceleration** as a function of velocity over time.

3. **Baseline Adjustments**

- Comparing values against an expected baseline helps highlight anomalies.
- Example: **Churn rate difference**, where customer retention is compared to an industry average.

4. **Normalization:**

- Following on from the previous case, normalization of parameter values based on the values of another parameter or baseline that is dynamically calculated given properties of other variables.
- One example here is *failed transaction rate*; in addition to looking at this value as a raw (or rescaled) count, it often makes sense to normalize this in terms of attempted transactions.

- Data scientists don't just use raw data; they create **new features** by combining different elements of existing data. These recombined features help build **better predictive models**.

Example: Customer Engagement Trend

- Suppose we track **customer engagement over time** (e.g., time spent on a website, purchase frequency).
- The **slope** of engagement (whether it is **increasing or declining**) is an important feature.
- However, **the impact of a small decline varies depending on the prior engagement level**:
 - A **highly engaged** user slightly reducing activity might not be a concern.
 - A **rarely engaged** user declining even more might indicate churn.

Reinterpreting Non-Numeric Features

- A key challenge in data processing is handling non-numeric features, as they often contain valuable information encoded in shorthand.
- For example, in stock trading, the stock identifier (e.g., AAPL) and the buyer/seller identities carry meaningful insights that affect trading behavior. Additionally, different stocks within the same industry may exhibit unique trading patterns, and internal company structures might influence transactions at different times.

1. Aggregation as a Basic Approach

- One straightforward way to handle non-numeric features is through aggregation. For instance, counting occurrences over time and analyzing changes in counts can be informative.
- **For Example:**
 - Instead of storing detailed logs of each purchase, the system **aggregates** the data by counting the **number of purchases per month** for each customer.
 - Example:
 - **Customer A:** 5 purchases in January, 3 in February, 7 in March.
 - **Customer B:** 2 purchases in January, 1 in February, 4 in March.

- However, aggregation risks reducing the information available to the model, increasing breakability and the likelihood of overfitting, particularly in deep learning models. Therefore, extensive aggregation is generally discouraged.

2.Encoding Non-Numeric Data

- A better alternative to aggregation is encoding categorical variables into numerical formats. One of the most widely used encoding techniques is **one-hot encoding**, which converts categorical data into binary variables. Each unique category becomes a separate feature, represented as either 0 or 1.
- **Example of One-Hot Encoding**

Consider a dataset with Age and Gender:

Case	Age	Gender
1	22	M
2	25	M
3	34	F
4	23	M
5	25	F
6	41	F

- After applying one-hot encoding, the dataset transforms into:

Case	Age_22	Age_23	Age_25	Age_34	Age_41	Gender_F	Gender_M
1	1	0	0	0	0	0	1
2	0	0	1	0	0	0	1
3	0	0	0	1	0	1	0
4	0	1	0	0	0	0	1
5	0	0	1	0	0	1	0
6	0	0	0	0	1	1	0

- One-hot encoding is beneficial because it retains detailed categorical information without reducing dataset granularity. Additionally, it helps in identifying important response categories while ignoring less meaningful ones.

3.The Hash Trick for Encoding Text Data

- Another method for converting non-numeric features into numerical data is **hashing**. Hash functions transform textual data into numerical representations. Hashes are commonly used for security and data compression, but they also serve as a powerful tool in feature engineering.
- A basic hash function assigns a numeric value to each character (e.g., 'a' = 1, 'b' = 2, etc.). For instance, the phrase "**cat gifs**" could be hashed as follows:
 - **Cat** → 3 + 1 + 20 = 24
 - **Gifs** → 7 + 9 + 6 + 19 = 41
 - **Total Hash Value** = 24 + 41 = 65
- However, this naive approach has drawbacks:
 1. **Unbounded Output** – The hash values can grow indefinitely, which contradicts the purpose of dimensionality reduction.
 2. **Small Changes Yield Small Differences** – If "cat" changes to "bat," the hash value changes only slightly (from 65 to 64), whereas a good hash function should create large variations in output for small input changes.

Improving Hashing for Feature Engineering

- A well-designed hash function ensures that:
 - Small changes in input cause significant variations in output.
 - The number of possible outputs is bounded (often using powers of two for efficiency).

The Hash Trick and Zipf's Law

- The **hash trick** leverages hashing to reduce dimensionality while preserving key information. A common issue with hashing is **hash collisions**, where different words generate the same hash. However, Zipf's Law—stating that word frequency follows a predictable pattern—helps mitigate this issue.

According to Zipf's Law:

- The probability of encountering the **n-th** most common term is approximately **0.1/n** (up to ~1,000 words).
- The majority of words in a dataset are infrequent, reducing the likelihood of collisions affecting model performance.

Thus, by using a well-designed hash function and leveraging the natural distribution of word frequency, the hash trick allows a massive reduction in feature dimensions without significant information loss.

Using feature selection techniques

- Feature selection is a fundamental process in machine learning that involves identifying the most important features while eliminating irrelevant or redundant ones. This improves model performance, reduces overfitting, and enhances interpretability.
- Challenges of Feature Selection as follows

1. The Impact of Uncontrolled Feature Addition

- When performing **feature engineering**, we may create **large and complex datasets** with many features. However, excessive feature addition can introduce **model fragility and overfitting**, particularly in the following cases:
 - **Simple distinctions between large groups:** A model can handle a **larger number of features** without a significant risk of overfitting.
 - **Complex distinctions or small group sizes:** Adding too many features **reduces classification consistency and effectiveness**, as the model may struggle to generalize patterns.

2. Feature Suitability Varies by Model Type

- Not all features contribute equally across different machine learning models.
 - **Decision forests (e.g., Random Forest, Gradient Boosted Trees):** These models do not necessarily benefit from transformations such as **log-scaling** because they work well with raw features.
 - **Linear models (e.g., Logistic Regression, Neural Networks):** These models require **scaling and transformation** to ensure optimal performance.
- Selecting and preprocessing features correctly is **critical** to achieving the best performance for a given algorithm.

3. Traditional Feature Selection Approach

- Historically, feature selection aimed to define a **mathematical function** that relates key inputs to the **desired outcome**.
 - **Feature count limitation:** Early methods included only essential features, treating additional parameters as **nuisance variables**.
 - **Overfitting risk:** Each new feature adds complexity to the model, increasing the risk of **memorization rather than generalization**.

Example of Overfitting

- **Using a unique identifier (e.g., customer ID) as a feature:** The model learns **specific cases** instead of recognizing general patterns, leading to poor performance on new data.
- **Separating cases into very small groups:** This leads to **poor generalization** as the model memorizes rather than learns.
- A traditional principle in feature selection suggests **starting with fewer features and gradually adding new ones** if they improve the model.

4. The Modern Brute-Force Approach to Feature Selection

- In contrast to traditional feature selection, modern machine learning often **starts with a large number of features and systematically reduces them** using automated selection techniques.

How Brute-Force Feature Selection Works

- **Exhaustive Feature Testing:** Models are trained using different feature subsets to determine the most effective combination.
- **Parallel Processing:** Modern systems enable feature selection **across large datasets**, handling **hundreds or thousands of features** efficiently.
- **Automation:** Algorithms automatically **eliminate weak or redundant features**, optimizing the feature set.

Advantages of Brute-Force Feature Selection

- Identifies **optimal feature combinations** efficiently.
- Reduces the need for **manual feature selection**.
- Can handle **high-dimensional datasets** effectively.

Disadvantages of Brute-Force Feature Selection

- **Black-box nature:** These techniques **do not explain why** certain features are selected, making them harder to interpret.
- **Over-reliance on automation:** Domain knowledge is still **essential** to validate feature selection results.

- **Popular Feature Selection Techniques**

- **Filter Methods** (based on statistical relationships)
 - Correlation Matrix
 - Mutual Information
 - Chi-Square Test
- **Wrapper Methods** (iterative model-based selection)
 - Recursive Feature Elimination (RFE)
 - Forward/Backward Feature Selection
- **Embedded Methods** (built into the model)
 - Lasso Regression (L1 regularization)
 - Ridge Regression(L2)
 - Decision Tree Feature Importances

Performing feature selection

- When working with a large dataset, the challenge often shifts from data collection to selecting the most relevant features for modeling. Feature selection helps in reducing the dimensionality of data while improving model performance and interpretability. This section explores various feature selection techniques, including:
 - **Correlation Analysis**
 - **Regularization Techniques (LASSO and Ridge Regression)**
 - **Recursive Feature Elimination (RFE)**
 - **Genetic Models**

1. Correlation Analysis: Detecting Multicollinearity

Understanding Multicollinearity

Multicollinearity refers to high correlation between independent variables, which can negatively impact regression models. For example, the number of pizza slices and the price of a pizza are highly correlated. This means one variable contains similar information to the other, making one of them redundant.

Types of Multicollinearity

1. Structural Multicollinearity

- Occurs when new features are created from existing ones.
- Example: If f_1 is derived from f_2 , they may be highly correlated.

2. Data-Based Multicollinearity

- Occurs when two variables are influenced by the same external factor.
- Example: Income and spending behavior may be correlated because of economic conditions.

Effects of Multicollinearity

- Leads to unstable model coefficients.
- Affects feature importance ranking.
- Reduces model interpretability.
- Causes overfitting in some cases.

Detecting Multicollinearity Using Eigenvalues

A useful technique to identify multicollinear features is **eigenvalue decomposition** of the correlation matrix. Eigenvalues quantify how much independent information each feature contributes. A small eigenvalue indicates redundancy.

Example: Identifying Multicollinear Features

```
import numpy as np
import pandas as pd
# Create a simple dataset
```

```

data = {
    "F1": [2, 3, 4, 5, 6],
    "F2": [1, 2, 3, 4, 5],
    "F3": [3, 6, 9, 12, 15], # 3 * F1 (high correlation)
    "F4": [4, 8, 12, 16, 20], # 4 * F1 (high correlation)
    "F5": [5, 10, 15, 20, 25], # 5 * F1 (high correlation)
    "Y": [10, 20, 30, 40, 50] # Target variable
}
df = pd.DataFrame(data)
# Compute correlation matrix
corr_matrix = df.corr()
# Display correlation matrix
print("Correlation Matrix:\n", corr_matrix)

```

Expected Output

```

Correlation Matrix:
      F1  F2  F3  F4  F5  Y
F1  1.00  1.00  1.00  1.00  1.00  1.00
F2  1.00  1.00  1.00  1.00  1.00  1.00
F3  1.00  1.00  1.00  1.00  1.00  1.00
F4  1.00  1.00  1.00  1.00  1.00  1.00
F5  1.00  1.00  1.00  1.00  1.00  1.00
Y   1.00  1.00  1.00  1.00  1.00  1.00

```

- All features are **highly correlated**, meaning we can **remove F3, F4, and F5** because they don't add unique information.

2. Regularization Techniques: LASSO and Ridge Regression

- Regularization methods are used to handle multicollinearity and perform feature selection.

LASSO (Least Absolute Shrinkage and Selection Operator)

- LASSO regression (L1 regularization) is a technique that adds a penalty term to the loss function to shrink less important feature coefficients to **zero**, effectively selecting only the most important features.
- **Mathematical Formula**
 - For a dataset with features X and target variable Y, the LASSO loss function is:

$$E(X, Y) + \alpha \sum_{i=1}^n |w_i|$$

Where:

- α is the regularization parameter that controls the strength of feature selection.
- w_i are the feature coefficients.

Limitations of LASSO

- It can be unstable when features are highly correlated.
- It may randomly select one feature from a correlated set instead of selecting the best.

Ridge Regression (L2 Regularization)

- Unlike LASSO, Ridge regression (L2 regularization) **does not eliminate features** but distributes the coefficients more evenly among correlated features. It is useful for reducing overfitting.
- **Mathematical Formula**

$$E(X, Y) + \alpha \sum_{i=1}^n w_i^2$$

- The squared penalty term prevents coefficients from becoming too large but does not shrink them to zero.

Advantages of Ridge Regression

- More **stable** than LASSO, especially when features are correlated.
- Preserves all features but reduces their impact.

Example:

```
import numpy as np
```

```

import pandas as pd
from sklearn.linear_model import Lasso, Ridge
from sklearn.preprocessing import StandardScaler
# Create a simple dataset
data = {
    "F1": [2, 3, 4, 5, 6],
    "F2": [1, 2, 3, 4, 5],
    "F3": [3, 6, 9, 12, 15], # 3 * F1 (high correlation)
    "F4": [4, 8, 12, 16, 20], # 4 * F1 (high correlation)
    "F5": [5, 10, 15, 20, 25], # 5 * F1 (high correlation)
    "Y": [10, 20, 30, 40, 50] # Target variable
}
df = pd.DataFrame(data)
# Separate features and target variable
X = df.drop(columns=["Y"])
Y = df["Y"]
# Standardize the dataset
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
### **LASSO Regression**
lasso = Lasso(alpha=0.1) # Alpha controls feature selection strength
lasso.fit(X_scaled, Y)

print("\nLASSO Selected Features:")
for name, coef in zip(X.columns, lasso.coef_):
    print(f'{name}: {coef:.3f}')

### **Ridge Regression**
ridge = Ridge(alpha=0.1) # Alpha controls regularization strength
ridge.fit(X_scaled, Y)

print("\nRidge Coefficients:")
for name, coef in zip(X.columns, ridge.coef_):
    print(f'{name}: {coef:.3f}')

```

Output:

LASSO Selected Features:

```

F1: 14.042
F2: 0.000
F3: 0.000
F4: 0.000
F5: 0.000

```

Ridge Coefficients:

```

F1: 2.817
F2: 2.817
F3: 2.817
F4: 2.817
F5: 2.817

```

3. Recursive Feature Elimination (RFE)

- **Recursive Feature Elimination (RFE)** is a wrapper method used to iteratively select the best features by eliminating the least important ones.
- **How RFE Works:**
 1. Start with all features.
 2. Train a model (e.g., **Logistic Regression, SVM, Decision Tree**).
 3. Rank features based on importance scores.
 4. Remove the least important feature.
 5. Repeat until the desired number of features remains.

Advantages of RFE:

- Works well with **any estimator** (SVM, Logistic Regression, Random Forest).
- Helps rank **feature importance**.
- Handles **both linear and non-linear models**.

Disadvantages of RFE:

- Computationally expensive for large datasets.
- Model-specific (performance depends on the chosen estimator).

Python Program for RFE Feature Selection

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression

# Create a linear regression model
model = LinearRegression()

# Apply Recursive Feature Elimination (RFE) to select top 2 features
rfe = RFE(model, n_features_to_select=2)
rfe.fit(X, Y)

# Print selected features
print("RFE Selected Features:")
for name, support in zip(X.columns, rfe.support_):
    if support:
        print(name)
```

Expected Output

```
RFE Selected Features:
F1
F2
```

4.Genetic Models in Feature Selection

Introduction to Genetic Algorithms

- Genetic algorithms (GAs) are optimization techniques inspired by natural selection.
- They are useful for feature selection in models with a large number of parameters.
- These algorithms iteratively refine feature subsets to improve model performance.

Process of Genetic Feature Selection

- 1. Initial Selection of Features**
 - A set of predictors (features) is randomly combined into multiple subsets (candidates).
 - A performance measure is calculated for each candidate subset.
- 2. Selection and Recombination**
 - Predictors from the best-performing candidates are recombined into a new generation.
 - Recombination involves mixing features from different candidates to form new models.
 - Each subset has a probability of mutation (adding or removing a feature).
- 3. Iteration Process**
 - The algorithm runs for multiple generations.
 - Performance improvement follows a nonlinear pattern:
 - Initially, performance improves rapidly.
 - Later, the rate of improvement slows down.
 - After a certain point, further iterations may lead to overfitting.
 - The optimal number of iterations is determined through training data analysis.
- 4. Finding the Best Iteration Count**
 - Testing is done using training data.
 - The Root Mean Squared Error (RMSE) is plotted against iteration count.
 - The optimal number of iterations is chosen based on the RMSE curve.

Detailed Steps in Each Generation

- 1. Candidate Creation**
 - First-generation candidates use a random sample of available predictors.
 - Typically, 50% to 80% of the available features are used initially.
 - A smaller percentage is used when the number of features is large.
- 2. Performance Evaluation (Fitness Measure)**
 - Performance is measured using cross-validation:
 - **Internal Cross-Validation:**

- Used within the model to track performance per iteration.
 - Fitness scores from internal validation help select models for the next generation.
 - **External Cross-Validation:**
 - Uses a separate dataset to check if the model has overfitted to training data.
3. **Recombination Process**
- Controlled by three key parameters:
 - **Mutation Probability:**
 - Defines the chance of random feature addition/removal in the next generation.
 - Helps maintain diversity and avoid local optima.
 - **Cross-over Probability:**
 - Defines the likelihood that two parent candidates will combine features.
 - Various cross-over techniques exist:
 - Splitting parent feature sets (first half/second half).
 - Randomly selecting features from both parents.
 - Using common features from both parents.
 - **Elitism (Optional):**
 - Preserves top-performing models from the current generation.
 - Ensures high-performing candidates are not lost.
 - These models may also undergo mutations or be used as parents for new models.

Application of Genetic Algorithms in Feature Selection

- Genetic algorithms can be used as wrappers for existing models like:
 - Logistic Regression
 - Support Vector Machines (SVM)
 - Other machine learning models
- They provide flexibility and adaptability across different quantitative fields.

Feature Engineering in Practice

Understanding the Need for Feature Engineering

- Depending on the modeling technique, the value of feature engineering varies.
- Deep learning models perform better with less-engineered data, while shallower models benefit from feature engineering.
- The key is to **iterate quickly** from dataset acquisition to modeling.
- The first pass should focus on the **minimum required processing** to achieve an acceptable model accuracy.
- Each iteration should provide **insights** to refine the feature engineering process.

A Practical Example: Predicting Commute Delays

- The example focuses on **improving commute experience** by predicting delays.
- The approach involves working with an **unfamiliar, high-dimensional dataset** iteratively to enhance modeling.
- The case study is based on **commuting in Vancouver**, where delays and congestion are unpredictable.

Steps to Improve the Commute Prediction Model

1. **Harvesting Data from Multiple APIs**
 - Includes **text-based and climate streams**.
2. **Applying Feature Engineering Techniques**
 - Deriving new variables from the collected data.
3. **Generating Commute Delay Risk Scores**
 - Testing the feature set by computing delay probabilities.

Focus on a Self-Sufficient and Generalizable Model

- Unlike traditional feature engineering, the goal is not just to build a high-performing model but to create a **self-adjustable solution**.
- The solution should be **applicable beyond Vancouver**, allowing customization for different localities.

Challenges in Using Twitter Data

- Twitter's **API terms of use** require any modifications (such as deleted tweets) to be reflected in extracted datasets.

- This makes it **impractical to share real Twitter data** in a public repository.
- As a result, users must **build their own Twitter data stream**.
- **Seasonal variations** and local context also affect model performance, making generalized solutions complex.

The Need for a General Solution

- The approach must be **adaptable to various geographic locations** and datasets.
- The code in the next section serves as a **base for further development**.
- It can be used as:
 - A **commercial application**.
 - A **personalized, data-driven life improvement tool**.

Acquiring Data via RESTful APIs

Requirements for Data Collection

- The dataset must be **rich and timestamped**.
- Data should be captured at a **sufficient frequency** (at least one record per commute period).
- This enables effective **model training**.

Using the Twitter API for Data Collection

- The Twitter API provides access to **recent tweet data**.
- It can be utilized in two primary ways:
 1. **Harvesting Official Transit Authority Tweets**
 - **Bus and train companies** provide real-time updates on **delays and service disruptions**.
 - These updates often follow a **consistent format**, making them easier to **tag and analyze**.
 2. **Capturing Commuter Sentiment**
 - Tweets can be filtered **geographically** for a specific area.
 - A **customized dictionary** can be used to identify **keywords related to disruptions**.
 - This helps in understanding **real-time commuter experiences**.

Additional Data Sources for Model Improvement

1. **Bing Traffic API**
 - Provides **traffic congestion and disruption incidents**.
 - Covers a **user-specified geographical area**.
 - Can be used to **enhance commute predictions**.
2. **Yahoo Weather API**
 - Returns **current weather data** based on a **zip code or location**.
 - Includes:
 - **Temperature**
 - **Wind speed**
 - **Humidity**
 - **Atmospheric pressure**
 - **Visibility**
 - Also provides:
 - A **textual description** of current conditions.
 - **Forecast data** for upcoming weather trends.

Testing the Performance of Our Model

To effectively evaluate our commute disruption prediction model, it is essential to establish test criteria and an appropriate performance score. The goal is to determine the likelihood of commute disruptions for the current day, ideally with enough advance notice to take preventive actions, such as leaving home earlier.

Key Considerations:

1. **Understanding Model Output:** The model is expected to generate a numerical score between 0 and 1, representing the risk of commute disruption.
2. **Quantifying Model Performance:** A relevant performance measure is required to assess the model.
3. **Target Data for Scoring:** A reference dataset is necessary for evaluating the model's accuracy.

Importance of Performance Testing:

- Some models primarily serve as information sources, providing insights that were previously unavailable. However, every model inherently has a performance criterion, even if it is subjective (such as user satisfaction). Measuring performance allows for iterative improvements and ensures the model remains effective. Therefore, performance testing should not be disregarded.

Model Output and Scaling:

- The model produces a numerical score (0-1) for outbound commutes each day.
- One approach to refining the score is log rescaling, considering that commute delay times may follow a power law.
- Initially, no transformations will be applied to the scores, awaiting an evaluation of the model's raw output.

Practical Interpretation:

- A numerical score between 0 and 1 may not be directly useful for decision-making.
- A bucketed system (e.g., high risk, medium risk, low risk) may provide better practical guidance.
- Transitioning to a categorical classification system should be considered after reviewing commute delay distributions.

Performance Measurement:

- The choice of performance metric depends on the problem characteristics.
- A confusion matrix provides insight into classification errors.

Prediction	Actual Result	Outcome Type	Impact
TRUE	TRUE	True Positive	Correctly predicts a delay.
FALSE	FALSE	True Negative	Correctly predicts no delay.
FALSE	TRUE	False Negative	Fails to predict a delay (unexpected delays occur).
TRUE	FALSE	False Positive	Predicts a delay when none occurs (unnecessary early departures).

- The four key contingencies are:
 - True Positive (correctly predicted delays)
 - True Negative (correctly predicted no delays)
 - False Positive (incorrectly predicted delays)
 - False Negative (missed commute delays)

Since both false positives and false negatives impact commute planning, a performance measure emphasizing sensitivity (true positive rate) and specificity (false positive rate) is required. The **Area Under the Curve (AUC)** is the most suitable metric in this scenario.

Target Data Collection:

- Commute times were self-recorded using a stopwatch, with a consistent start time and route.
- Limitations of this approach:
 - Personal biases (e.g., morning sluggishness, localized trends) may affect data.
 - A more comprehensive dataset from multiple commuters and routes would be preferable.
- However, since the objective is to predict personal commute disruptions, the data remains useful.

Defining a Baseline:

- A baseline model assumes that commute delays do not occur.
- This aligns with typical human behavior—acting as if transit disruptions will not happen.
- Out of 85 recorded cases, 14 commute delays were observed.
- The baseline model's target AUC score to surpass is **0.5**.

Twitter Data Analysis for Vancouver's Translink Service Announcements

Introduction

- The analysis focuses on Vancouver and utilizes **Translink's Twitter data** as an additional data source.
- **Translink Twitter** posts structured service announcements, making it ideal for text mining and further analysis.
- The **Twitter API** is used to collect tweets over an extended period, extracting **date** and **body text** from each tweet.

Tweet Content and Structure

- Translink tweets contain key information such as:
 - **Nature of the tweet** (delay or non-delay)
 - **Affected station**
 - **Cause of delay**
- Translink tweets cover both **SkyTrain** and **bus routes** but maintain a consistent structure, using **hashtags** to distinguish them:
 - **#RiderAlert** – Bus route information

- **#SkyTrain** – Train-related information
- **#TransitAlert** – General alerts (e.g., holidays)
- Specific keywords identify different types of service issues:
 - **Delay** – Service delay
 - **Detour** – Route change
 - **Diversion** – Service redirection

Data Extraction and Cleaning

- The **translink_tweet_data.json** file contains collected tweet data.
- A **scraper script** is provided in the GitHub repository to collect tweets, requiring a **Twitter Developer account**.
- Cleaning and text preprocessing are applied:
 - **BeautifulSoup** extracts text from tweets.
 - **NLTK (Natural Language Toolkit)** is used for text processing and tagging.

Code Example for Cleaning and Tagging

```
from bs4 import BeautifulSoup
tweets = BeautifulSoup(train["TranslinkTweets.text"])
tweettext = tweets.get_text()

import nltk
from nltk.tag import NgramTagger
brown_a = nltk.corpus.brown.tagged_sents(categories='a')
tagger = None
for n in range(1,4):
    tagger = NgramTagger(n, brown_a, backoff=tagger)
taggedtweettext = tagger.tag(tweettext)
```

- **Key Steps:**
 - **Extract text** from tweets using BeautifulSoup.
 - **Tag words** using **NgramTagger** based on the Brown corpus.

Feature Engineering

- Service disruptions are categorized based on:
 - **Location** – Street or station name.
 - **Time** – Timestamp of the tweet.
 - **Cause of delay** – Standardized terms.
- **Challenges in Data Quality:**
 - **Location** is given as a station or street name, requiring further processing.
 - **Time** is inferred from the tweet's timestamp, which may have delays in reporting.
 - **Causes of delay** are consistently mentioned in tweets, making them reliable features.

Delay Causes for Different Services

SkyTrain Delay Categories:

1. **Rail**
 2. **Train**
 3. **Switch**
 4. **Control**
 5. **Unknown**
 6. **Intrusion**
 7. **Medical**
 8. **Police**
 9. **Power**
- **Police, Power, and Medical delays** may not be relevant for modeling service delays.
 - Failures related to **train, track, and switch** could correlate with bus detours.

Bus Route Delay Categories:

1. **Motor Vehicle Accident (MVA)**
 2. **Construction**
 3. **Fire**
 4. **Watermain**
 5. **Traffic**
- Encoding these categories helps in predictive analysis.

One-Hot Encoding for Delay Causes

- **Objective:** Convert delay categories into numerical features.
- **Steps:**
 - Create a **binary variable** for each delay type (0 or 1).
 - **Check tweet content** for delay-related terms.
 - Assign **1** if the tweet contains the specific delay type, otherwise **0**.

Code Example for One-Hot Encoding

```
from sklearn import preprocessing

enc = preprocessing.OneHotEncoder(categorical_features='all', dtype='float',
                                  handle_unknown='error', n_values='auto', sparse=True)
```

```
tweets.delayencode = enc.transform(tweets.delaytype).toarray()
```

- **Creates categorical variables** for different types of delays.
- **Encodes them into numerical format** for machine learning models.

Analyzing Disruption Frequency and Risk

- **Hypothesis:**
 - **Multiple disruptions** in a short period might indicate a higher probability of future disruptions.
- **Approach:**
 - Start with **basic features** before introducing complex ones.
 - Focus on **initial incident rates** before adding additional patterns.

Consumer Comments on Twitter

- **Public Twitter data** can provide **timestamped** reports on transit and road conditions.
- **Approach:**
 - Use **dictionary-based keyword search** to find tweets mentioning delays.
 - Define **bounding box coordinates** to limit the search to Vancouver.

Processing Consumer Comments

- **Challenges:**
 - Users do not follow a **consistent format**, unlike Translink.
 - Requires **text cleaning** and **normalization** similar to Translink tweets.
- **Future Enhancements:**
 - Create **specialized dictionaries** for identifying causes (e.g., construction-related delays).

Bing Traffic API

Introduction

- The **Bing Traffic API** provides **free access** to traffic incident data.
- Unlike some competitor APIs that are **behind paywalls**, this API offers **detailed information** on traffic incidents.
- The API returns various attributes, including:
 - **Incident location code**
 - **General description of the incident**
 - **Congestion information**
 - **Incident type code**
 - **Start and end timestamps**

Incident Type Codes

- The API categorizes traffic incidents into **11 types**:
 1. **Accident**
 2. **Congestion**
 3. **DisabledVehicle**
 4. **MassTransit**
 5. **Miscellaneous**
 6. **OtherNews**
 7. **PlannedEvent**
 8. **RoadHazard**
 9. **Construction**
 10. **Alert**
 11. **Weather**

Severity Codes

- Incidents are also assigned a **severity code**, categorized as:

1. **LowImpact**
2. **Minor**
3. **Moderate**
4. **Serious**

Limitations of the API

- **Inconsistency across regions:** The API does not return the same information across different locations.
- **Example:**
 - **France:** Observed only **codes 1, 3, 5, and 8** over a **one-month period** in northern France.
 - **Vancouver:** Data is mainly limited to **codes 9 (Construction) and 5 (Miscellaneous)**.
 - Even "**Miscellaneous**" incidents in Vancouver are still **construction-related**.
- **Example Tweet:**

Closed between Victoria Dr and Commercial Dr - Closed. Construction work. Type: 5
- **No workaround for missing data** unless a **paid dataset** is used.

Querying the API – Example Code

Python Implementation

```
import urllib.request, urllib.error, urllib.parse
import json

latN = str(49.310911)
latS = str(49.201444)
lonW = str(-123.225544)
lonE = str(-122.903931)

url = ('http://dev.virtualearth.net/REST/v1/Traffic/Incidents/'
      + latS + ',' + lonW + ',' + latN + ',' + lonE +
      '?key=GETYOUROWNKEYPLEASE')

response = urllib.request.urlopen(url).read()
data = json.loads(response.decode('utf8'))
resources = data['resourceSets'][0]['resources']

print('-----')
print('PRETTIFIED RESULTS')
print('-----')

for resourceItem in resources:
    description = resourceItem['description']
    typeof = resourceItem['type']
    start = resourceItem['start']
    end = resourceItem['end']

    print('description:', description)
    print('type:', typeof)
    print('starttime:', start)
    print('endtime:', end)
    print('-----')
```

API Response Example

vbnet

CopyEdit

PRETTIFIED RESULTS

```
description: Closed between Boundary Rd and Pierview Cres - Closed due to roadwork.
type: 9
severity: 4
starttime: /Date(1458331200000)/
endtime: /Date(1466283600000)/
```

description: Closed between Commercial Dr and Victoria Dr - Closed due to roadwork.
type: 9
severity: 4
starttime: /Date(1458327600000)/
endtime: /Date(1483218000000)/

description: Closed between Victoria Dr and Commercial Dr - Closed. Construction work.
type: 5
severity: 4
starttime: /Date(1461780543000)/
endtime: /Date(1481875140000)/

description: At Thurlow St - Roadwork.
type: 9
severity: 3
starttime: /Date(1461780537000)/
endtime: /Date(1504112400000)/

Key Observations

- The API **successfully retrieves** traffic incidents with:
 - **Incident description**
 - **Type of incident**
 - **Severity level**
 - **Start and end times**
- **Issue:** Some locations **do not return all incident types**.
- **Example:** Vancouver mostly shows **Construction (9)** and **Miscellaneous (5)**, limiting its usefulness

Deriving and Selecting Variables Using Feature Engineering Techniques

- **Initial Feature Selection:**
 - In the first pass, a small feature set was chosen despite many available opportunities in the data.
 - The priority was to obtain initial results quickly rather than exploring all possible features.
 - The initial dataset might not effectively solve the problem or meet target expectations.
 - Iterative improvement of the feature set is necessary by creating new features and reducing the number of less valuable ones.
- **Feature Engineering Process:**
 - Use techniques such as one-hot encoding and Recursive Feature Elimination (RFE).
 - One-hot encoding is applied to weather data and tweet dictionaries, creating m*n-sized tensors.
 - RFE helps in reducing misleading or redundant features, especially when multiple features reinforce the same signal.
 - Some misleading terms may persist despite prior data cleaning efforts
- **Expand-Contract Methodology:**
 - Expansion: Generate potentially valuable features through transformations and encodings.
 - Contraction: Identify and retain the most performant subset while removing underperforming features.
 - Test different target feature counts to determine the best available feature set.
- **Different Approaches to Feature Engineering:**
 - Some data scientists build all features first and then reduce them later to avoid data loss.
 - Others perform feature creation and reduction iteratively.
 - The choice of methodology depends on individual preferences and project requirements.
- **Initial Dataset Structure:**

```
{  
  'DisruptionInformation': {  
    'Date': '15-05-2015',  
    'TranslinkTwitter': [  
      {'Service': '0', 'DisruptionIncidentCount': '4'},
```



```

        {'Service': '1', 'DisruptionIncidentCount': '0'}
    ]
},
'BingTrafficAPI': {
    'NewIncidentCount': '1',
    'SevereIncidentCount': '1',
    'IncidentCount': '3'
},
'ConsumerTwitter': {
    'DisruptionTweetCount': '4'}
}

```

- This dataset is unlikely to perform well but serves as a starting point.

- **Initial Algorithm Implementation:**

- A simple linear regression model was chosen for the first iteration.
- Linear regression provides a fast, transparent way to assess feature performance.
- Initial Code:

```

from sklearn import linear_model
tweets_X_train = tweets_X[:-20]
tweets_X_test = tweets_X[-20:]
tweets_y_train = tweets.target[:-20]
tweets_y_test = tweets.target[-20:]

regr = linear_model.LinearRegression()
regr.fit(tweets_X_train, tweets_y_train)

print('Coefficients: \n', regr.coef_)
print("Residual sum of squares: %.2f" % np.mean((regr.predict(tweets_X_test) -
tweets_y_test) ** 2))
print('Variance score: %.2f' % regr.score(tweets_X_test, tweets_y_test))

plt.scatter(tweets_X_test, tweets_y_test, color='black')
plt.plot(tweets_X_test, regr.predict(tweets_X_test), color='blue', linewidth=3)
plt.xticks(())
plt.yticks(())
plt.show()

```

- Initial results show a poor AUC of 0.495, performing worse than the target.

- **Confusion Matrix Analysis:**

- Model fails to predict disruptions accurately.
- The incidence rate suggests disruptions occur every 16 commutes (6% occurrence rate).
- A better strategy might be to assume disruptions happen daily rather than relying on this model.

- **Improvement Strategies:**

- Enhance input data further by creating new features using transformation techniques.
- Extend the dataset with additional information, such as weather data (temperature and humidity).
- Consider upgrading the algorithm (e.g., using Random Forest or SVM) but only after maximizing feature engineering benefits.

- **Feature Encoding for Location Data:**

- Location encoding can be done in two ways:
 1. Parse input data using a corpus of street names/locations to build a one-hot matrix.
 2. Apply one-hot encoding directly to tweets and API data.
- If dimensionality reduction techniques are used, redundant features will be removed via RFE.

- **Implementation of Location Encoding:**

```

from sklearn import preprocessing
enc = preprocessing.OneHotEncoder(categorical_features='all', dtype='float',
handle_unknown='error', n_values='auto', sparse=True)
tweets.delayencode = enc.transform(tweets.location).toarray()

```

- **Recent Count Variables from Translink and Bing Maps:**
 - Code for aggregation is available in the accompanying GitHub repository.
 - Rerunning the model with updated data showed a slight improvement.
 - Variance score increased to 0.56, indicating progress in the right direction.

Weather API and Commute Disruptions

1. Identifying Causes of Delays

- Previously, data sources were used to detect existing commute disruptions.
- Now, the focus is on identifying **causes** of these disruptions.
- **Weather conditions** are anecdotally linked to increased commute time.
- **Clear impacts:** Heavy frost, high winds, and severe weather events.
- **Unclear impacts:** Other weather patterns where the strength and nature of the effect are uncertain.

2. Extracting Weather Data

- Requires a weather data source with **high granularity** and **geo coverage**.
- **Yahoo Weather API** is chosen for extracting weather data.
- Provides a range of **temperature, atmospheric, and pressure-related** data.
- No API key or login is needed for access.

3. Querying the Yahoo Weather API

- Uses **YQL (Yahoo Query Language)** to fetch weather data.
- Example Python script:

```
import urllib2, urllib, json

baseurl = "https://query.yahooapis.com/v1/public/yql?"
yql_query = "select item.condition from weather.forecast where woeid=9807"
yql_url = baseurl + urllib.urlencode({'q': yql_query}) + "&format=json"
result = urllib2.urlopen(yql_url).read()
data = json.loads(result)
print data['query']['results']
```

- woeid=9807 is a **location identifier** (can be changed for different locations).
- Using item.condition retrieves current weather conditions.
- Using * retrieves **all available weather data**.

4. Example API Response

- **Current Weather Conditions:**

```
json
CopyEdit
{
  "channel": {
    "item": {
      "condition": {
        "date": "Thu, 14 May 2015 03:00 AM PDT",
        "text": "Cloudy",
        "code": "26",
        "temp": "46"
      }
    }
  }
}
```

- **7-Day Forecast Example:**

```
json
CopyEdit
{
  "item": {
    "forecast": {
      "code": "39",
      "text": "Scattered Showers",
      "high": "60",
      "low": "44",
      "date": "16 May 2015",
      "day": "Sat"
    }
  }
}
```

- **Other Weather Information:**

```

json
CopyEdit
{
  "astronomy": {
    "sunset": "8:30 pm",
    "sunrise": "5:36 am"
  },
  "wind": {
    "direction": "270",
    "speed": "4",
    "chill": "

```

Extracting and Processing Data

- Data collected daily from May 2015 to January 2016.
- Forecast data may not be useful for real-time commute prediction.
- Important variables for modeling:
 - wind.direction
 - wind.speed
 - wind.chill
 - condition.temperature
 - condition.text

One-Hot Encoding for Weather Conditions

- Weather tags can be encoded as categorical variables.
- This significantly increases the feature set.
- Example of structured data with weather conditions:

```

{
  'DisruptionInformation': {
    'Date': '15-05-2015',
    'TranslinkTwitter': [
      {'Service': '0', 'DisruptionIncidentCount': '4'},
      {'Service': '1', 'DisruptionIncidentCount': '0'}
    ]
  },
  'BingTrafficAPI': {
    'NewIncidentCount': '1',
    'SevereIncidentCount': '1',
    'IncidentCount': '3'
  },
  'ConsumerTwitter': {
    'DisruptionTweetCount': '4'
  },
  'YahooWeather': {
    'temp': '45',
    'cloudy': '1',
    'thunderstorms': '0',
    'rain': '0',
    'foggy': '0',
    'sunny': '0'
  }
}

```

Data Transformations

Cross-Column Transformations

- Transforming features based on multiple columns.
- Example: Calculating duration from start and end dates.

- Most machine learning models naturally capture variable relationships.
- Deep learning models can better encode nonlinear relationships.

Cross-Row Transformations

- Typically involves aggregation across rows.
- Example: Using the average of the last n values.
- Example: Change in temperature over n hours is more informative than current temperature.
- Useful for predicting future commute conditions.

Model Performance and Evaluation

- Re-running the model with weather data improved performance.
- AUC score increased to 0.534.